

Modeling and Analysis of Cooperative Packet Recovery Protocol

Naeem, Muhammad; Atif, Muhammad; Ali, Arshad; Gulzar, Maryam; Hasrat, Imran Riaz

Published in:
IEEE Access

DOI (link to publication from Publisher):
[10.1109/ACCESS.2024.3389738](https://doi.org/10.1109/ACCESS.2024.3389738)

Creative Commons License
CC BY 4.0

Publication date:
2024

Document Version
Publisher's PDF, also known as Version of record

[Link to publication from Aalborg University](#)

Citation for published version (APA):
Naeem, M., Atif, M., Ali, A., Gulzar, M., & Hasrat, I. R. (2024). Modeling and Analysis of Cooperative Packet Recovery Protocol. *IEEE Access*, 12, 56334-56343. <https://doi.org/10.1109/ACCESS.2024.3389738>

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal -

Take down policy

If you believe that this document breaches copyright please contact us at vbn@aub.aau.dk providing details, and we will remove access to the work immediately and investigate your claim.

RESEARCH ARTICLE

Modeling and Analysis of Cooperative Packet Recovery Protocol

MUHAMMAD NAEEM¹, MUHAMMAD ATIF², (Senior Member, IEEE), ARSHAD ALI³,
MARYAM GULZAR⁴, AND IMRAN RIAZ HASRAT¹

¹Department of Computer Science, Aalborg University, 9220 Aalborg, Denmark

²Department of Computer Science and IT, The University of Lahore, Lahore 54590, Pakistan

³FAST School of Computing, National University of Computer and Emerging Sciences, Lahore 54770, Pakistan

⁴SE Department, LUT University, 53850 Lappeenranta, Finland

Corresponding author: Maryam Gulzar (Maryam.Gulzar@lut.fi)

ABSTRACT Real-time audio/video transmission through Internet media is an important part of communication. Due to bandwidth limitations and a noisy environment, delivery of multimedia content to a remote location is not 100% guaranteed. These limitations are the basic cause of missing packets which affect the Quality of Service (QoS). A protocol for the recovery of lost packets is described in [Maxemchuk, Nicholas F., K. Padmanabhan, and S. Lo. "A cooperative packet recovery protocol for multicast video." Network Protocols, 1997. Proc. of 1997 International Conference on. IEEE]. This protocol claims significant improvement in QoS. We formally specify the protocol in a network of timed automata. By model-checking (A mathematical technique), we find that packet recovery is not always there. In this article, We report such scenarios of malfunctioning in the protocol when the size of multimedia contents is known (e.g., live video/audio broadcasting) and middle-level servers have different rates of data sending and receiving. We formulate the effect of inter-packet delay and transmission speed difference on a buffer.

INDEX TERMS Distributed algorithm, model checking, formal specification, linear temporal logic, video distribution, packet recovery.

I. INTRODUCTION

There is a massive increase in video communication over the Internet because of the high data rate [1]. The primary concern is quality of service (QoS), particularly in critical systems [2], [3], [4], and [5], where a little mistake may cause big problems. Heterogeneity in communication networks reduces QoS in multimedia communication. There are several challenges in designing communication protocols to achieve a good QoS, for example, bandwidth limitation, massive network traffic, packet congestion, and many more. Packet loss in communication is a significant factor for QoS. Packet Loss Ratio (PLR) must be less than 1% for achieving good QoS in video conferencing [6].

Cooperative packet recovery protocol introduces a server with a broadcasting source for recovery of lost packets by a client [7]. The server stores packets in a buffer

and sends repair packets to the client when it receives a NACK request for a packet. There are many protocols for packet recovery in a live transmission. In cooperative packet recovery protocol [7], Maxemchuk et al. claim improvement in QoS by introducing a new packet recovery algorithm for multimedia (audio and video) broadcasting [8].

In this article, our main objective is to verify the functional requirements of the cooperative packet recovery protocol using formal specification and verification. Formal verification is a mathematical analysis method to measure the correctness of communicating systems.

There are several automatic verification and modelling tools [9], [10], [11], and [12], for verification of the real-time and hybrid systems [13], [14], and [15]. UPPAAL is an integrating tool environment for verification, modeling, and validation of real systems [9]. We need broadcast channels to model a protocol in multicast data transmission protocol. Using the UPPAAL toolset, we can make models of data broadcasting and synchronization of communicating

The associate editor coordinating the review of this manuscript and approving it for publication was Tariq Umer¹.

processes. This tool is successfully applied for formal verification of real-time systems [16], [17], and [18]. UPPAAL simplifies the process of system specification and modeling by providing a user-friendly graphical interface.

Contributions of this study: We design a time automata model for all processes of the cooperative packet recovery protocol using the UPPAAL toolset. We verify the protocol model for all formal requirements using a verifier (A feature of the UPPAAL toolset). In the results, we describe some formulas that do not satisfy the function requirements of the protocol. We also describe the violation scenarios of the requirements. We formulate the effect of transmission (sending and receiving) speed difference and inter-packet delay (IPD) on the active part of the buffer. We can find the optimized level of buffer size and inter-packet delay for reliable communication with high QoS.

In section II, we describe the formal verification of some real-time systems. In Section III, we describe the cooperative packet recovery protocol to understand the basic structure of the protocol. In Section IV, we describe the automata model of each participant of the cooperative packet recovery protocol. In Section V, we describe the specification difference of the extended architecture of the protocol. In Section VI, we formalise the functional requirements. In Section VII, describe the results after model checking of the protocol. In Section VIII, we describe the effect of transmission and receiving speed difference on the active part of the buffer. In Section IX, we describe the limitations and challenges faced in our research. In Section X, we conclude this article.

II. RELATED WORK

UPPAAL toolset is successfully applied for formal verification of some real-time systems [13], [14], [15], [16], [17], and [18].

In [14], Bordbar et al. describe the importance of verifying and validating a system model to ensure the QoS requirements and related issues. They evaluate the QoS of the wireless communication systems by conducting a case study of a PAN network. They highlight the requirements of wireless systems in a dynamic environment by carefully investigating QoS management to formally analyse the system using the UPPAAL toolset. They describe the basic mechanism and architecture for the management of QoS.

Atif et al. formally analyse various variants of heartbeat protocols such as dynamic heartbeat protocol, periodic heartbeat protocol, and expanding heartbeat protocol [13] and [16]. In this work, they design a timed automata model for the protocols using the UPPAAL toolset and formalise and verify the natural properties of the protocol. They highlight different functional requirements that are unsatisfactory and report counter examples as evidence of requirement failure.

In [17], Kartal et al. describe the conversion of timed input/output automata (TIOA) with timed automata of UPPAAL and describe the conditions for TIOA for the suitable conversion for verification. For this purpose, the authors develop a TUConvert algorithm and formally verify

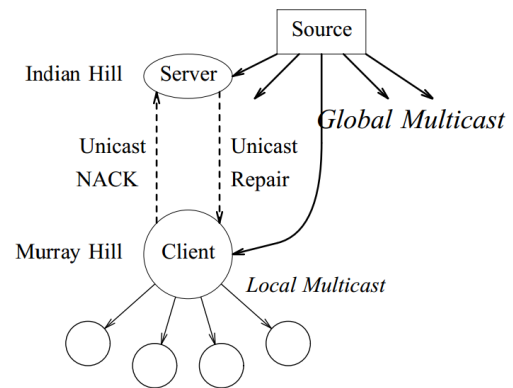


FIGURE 1. Basic architecture of the protocol [7].

the algorithm of conversation and the correctness of the algorithm using the UPPAAL toolset.

In [18], Ansgar et al. describe the verification of the medium access control protocol for wireless sensor networks using model checking technique. The authors use the UPPAAL toolset to verify the protocol. The aim of this work is to detect and resolve the collision problem in the protocol. The simulation result shows the efficiency of the proposed algorithm for collision avoidance.

III. COOPERATIVE PACKET RECOVERY PROTOCOL

A source, a server, a client and some receivers are the basic structural components of the protocol. The basic architecture of the packet recovery protocol is shown in Figure 1. In this protocol, the source broadcasts packets for the client and the server. The server is called the “retransmit server” because it transmits a packet if the client requests a lost packet. All packets have their sequence numbers in which the source broadcasts them. The server receives packets from the source and stores them in its buffer. According to the protocol [7], it is least likely that the server misses a packet sent by the source. The client also receives packets from the source and stores them in the proper place in its buffer. If the client receives a packet with a higher sequence number than expected, then it sends a NACK request to the server to recover the missing packet(s). After receiving packets from the source in a proper sequence, the client broadcasts packets to the receivers in a proper sequence. According to [7], the basic architecture was practically implemented between Indian Hill, Chicago and Murray Hill, New Jersey as shown in Figure 1. Live multimedia transmission was started from Indian Hill for the Murray Hill viewer.

The basic architecture is further extended into a large network. There is a number of receivers located in different areas, so there is a number of clients for each area, as shown in Figure 2. In extended architecture, the server is modified to cater to multiple clients instead of a single client. The server receives multiple NACK requests from the clients and stores them in a queue and responds to them in the same sequence.

There are two different architectures based on the communication channels between the server and clients.

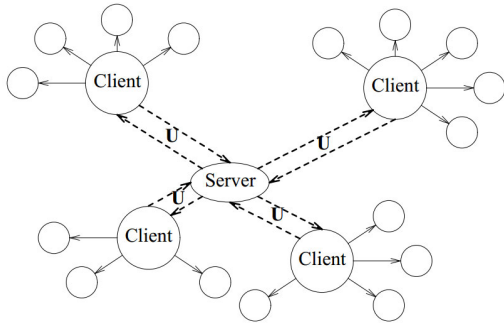


FIGURE 2. Architecture Configuration for Multiple clients with unicast channel [7].

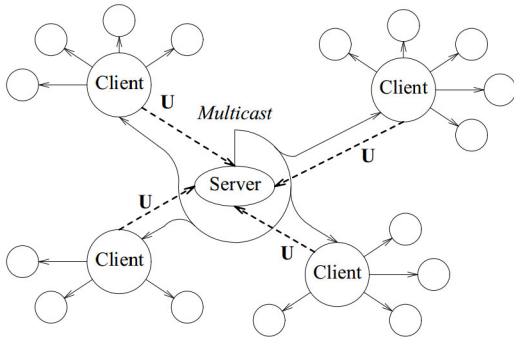


FIGURE 3. Architecture Configuration for Multiple clients with multicast channel [7].

In architecture 1 shown in Figure 2, the unicast channel is used for both communication channels between the client and the server; the communication in the extended architecture is improved by using a multicast channel from the server to the clients as shown in Figure 3. Architecture 2 is helpful in a scenario when a packet is missed by many clients and to recover their packets, the server use multicast instead of unicast.

IV. FORMAL SPECIFICATION OF THE COOPERATIVE PACKET RECOVERY PROTOCOL

We use the UPPAAL toolset for the formal specification of the protocol. UPPAAL is a model-checking toolset for the verification of distributed real-time systems. There is a graphical user interface to specify a process formally. We can verify the functional requirements by behavioral simulation of concurrent processes. There are two essential elements, a location and an edge (so-called transition), to specify states and actions, respectively. A circle represents a location; if it is initial state, it is a double-lined circle. A state can also be committed if the actions from this state are without interleaving the action from other states that are not committed. In other words, actions from a committed state get priority w.r.t. non-committed states. So, an action from a committed state/location must be taken to avoid deadlock. The purpose of an edge is to take some action where taking action can be conditional, i.e., a guarded action. During an action, values, and communication can be updated between

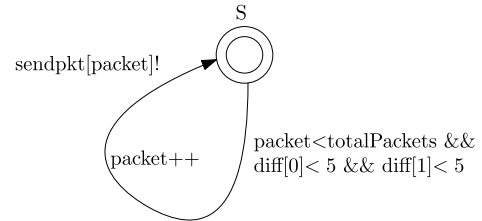


FIGURE 4. Formal specification of the source process.

TABLE 1. Declarations in formal specification of the client process.

<i>NackRecords</i>	This array maintains the NACK request record for all missing packets.
<i>retry_count</i>	This array maintains the number of NACK requests sent for a lost packet.
<i>front and rear</i>	These variables indicate the active part of the client's buffer. The rear indicates the packet to send, and the front indicates the next packet to receive.
<i>packet2Recover</i>	It indicates a lost packet to recover from the server.
<i>q[bufferSize]</i>	It is a client's buffer to store packets.
<i>isNack2Send</i>	It indicates the lost packet(s) in the buffer. It is <i>true</i> if any packet is missing in the buffer else <i>false</i> .
<i>send2Receivers</i>	It indicates the packets availability in the buffer for broadcasting.

process(es) [9]. Processes use channels for communication. A sender and a receiver processes use ‘!’ and ‘?’ signs, respectively, with the relevant channel name.

A. FORMAL SPECIFICATION OF THE SOURCE PROCESS

The source process specification is shown in Figure 4. There is a state with one edge. The edge is guarded with the condition ($diff < 5$) to control the packet-sending process to avoid the state-space explosion problem. The source stops sending packets when there is no more space for receiving new packets in the playback buffer of a client. The source process is synchronised with the server and the clients for sending packets via *SendPkt* (broadcast channel). The source process continuously multicast packets until it reaches the packet counter limit (end of transmission).

B. FORMAL SPECIFICATION OF THE CLIENT PROCESS

The formal Specification of the client process is shown in Figure 5. There are six edges in the client process model. To elaborate the Figure 5, we show the transitions by labelling the edges of the client process as *T1*, *T2*, *T3*, *T4*, *T5* and *T6*. There are two states: *initial* and *Sending2Receivers*. The first state is the initial state of the client process, and it takes most of the action (edges) from this state. As shown in Figure 5, *emSending2Receivers* is a committed state because the client must complete the *T6* edge to transmit packets before taking another edge. In Table 1, we describe the variables used in the specification of the client process:

We describe the edges of the client process as follows;

1) T1 TRANSITION

In this transition, the client process receives packets from the source via *SendPkt?* broadcast channel. It updates the variables involved in the behavioral activities of receiving

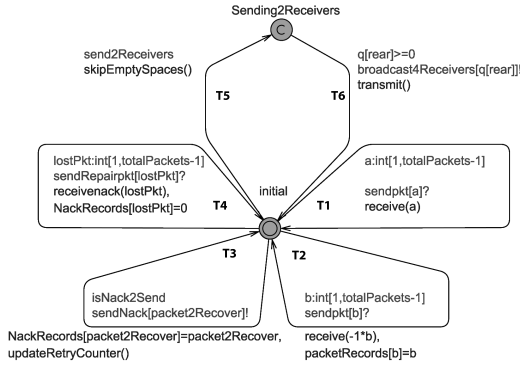


FIGURE 5. Formal Specification of the client process.

packets. It calls a function *Receive()* (described below) to update the variables of the client process.

```
void receive(int pkt){ // pkt shows received packet
    q[front]=pkt;
    front++;
    front = front%bufferSize;
    diff[cid]=diff[cid]+1; //cid shows client ID
    if (pkt>=0) {
        send2Receivers=true;
        if (pkt-1<last_received_pkt) {
            last_received_pkt = pkt;
            nextnack();
        }
        if (pkt>last_received_pkt)
            last_received_pkt = pkt;
        if (pkt<=0 && diff[cid]==5 &&
            send2Receivers==false ) {
            q[rear]=0; rear++;
            rear = rear%bufferSize;
            diff[cid]=diff[cid]-1;
        }
    }
}
```

Receive() performs the following functionality.

- The client adds received packets into the circular buffer *q* and updates *front* pointer of the buffer.
- *send2Receiver* is set to *true*, when it receives a packet from the source.
- *Nack2send* is set to *true* when it receives a packet with a greater sequence number than expected and checks for the recovery of lost packets by calling *nextnack()* function (described below).
- The client removes the oldest entry from its buffer to receive a packet when there is no more space.

```
void nextnack() {
    int [0,5]i=0;
    i=rear;
    while (q[i]<last_received_pkt &&
            isNack2Send==false) {
        if (q[i]<0 && (-1 x q[i])< last_received_pkt) {
            packet2Recover = -1 x q[i];
            isNack2Send=true;
            i=(last_received_pkt % bufferSize)-2;
            if (i<(bufferSize-1)) i++; else i=0;
        }
    }
}
```

nextnack() performs following functionality.

- It checks all packets in the buffer from *rear* pointer to *front* pointer when *isNack2Send* is *false* and set to *true* if a lost packet is pointed out in the buffer. The lost packet is assigned to the *packet2Recover* variable.

2) T2 TRANSITION

In *T2*, the source sends packets, but it is not delivered to the client. The client traces the lost packet and stores that packet in the buffer with a negative sequence number. *T2* calls *receive()* function (described above in *T1*) and sends a negative sequence number packet as an argument in the *receive()* function. It also adds lost packets record in *PacketRecord* array.

3) T3 TRANSITION

In *T3*, the client process sends NACK requests to the server via *SendNack* (unicast channel) to recover a lost packet. *T3* is guarded with *isNack2Send* (Boolean variable) and it is set to *true* when there is some lost packet(s) in the buffer. The client maintains NACK record to verify the successful recovery of all sent NACK requests for lost packets. It calls *UpdateRetryCounter()* function (described below) to maintain the number of requests sent for a particular packet.

```
void UpdateRetryCounter() {
    retry_count[(packet2Recover-1)%bufferSize]++;
    if (retry_count[(packet2Recover-1)%bufferSize]==3) {
        retry_count[(packet2Recover-1)%bufferSize]=0;
        q[(packet2Recover-1)%bufferSize]=0;
        isNack2Send=false;
        nextnack();
    }
}
```

UpdateRetryCounter() function performs the following set of actions.

- It increases *retrycount* for the requested packet when a NACK request is sent.
- It removes a packet and its record when a retry count equals three, and the lost packet is not recovered yet. *isNack2Send* variable is set to *false* and calls *nextnack()* function (Described above in *T1*), to check lost packet(s) in the buffer.

4) T4 TRANSITION

In *T4*, the client receives repaired packets from the server via *RepairPkt* channel. It removes the NACK record of a lost packet on successful recovery and updates the playback buffer with the repaired packet by calling *receiveack()* function (Described below).

```
void receiveack(int pkt) {
    //pkt shows received packet
    if (q[(pkt-1)%bufferSize]==(-pkt)) {
        q[(pkt-1)%bufferSize]=pkt;
        isNack2Send=false;
        Send2Receivers=true;
        if (pkt>last_received_pkt) last_received_pkt=pkt;
        nextnack();
    } else if (q[(pkt-1)%bufferSize]!=(pkt))
        Recovered_pkt_lost=1;
}
```

receiveack() function performs the following set of actions.

- It adds received packets into the buffer at proper location, to maintain the sequence.
- *isNack2Send* variable is set to *false* because we recovered a lost packet and *Send2Receiver* is set to *true* because now there is a packet in the buffer to broadcast.

- This function calls *nextnack ()* function (Discussed above in T1), to check the presence of any other lost packets.
- *Recovered_pkt_lost* is set to true when a recovered packet is not added in the buffer.

5) T5 TRANSITION

T5 and *T6* are used to send packets to the receivers. *T5* executes when there are packets in the buffer to transmit. In *T5*, the client removes all empty spaces from the buffer and moves to the next *Send2Receiver* committed state to send packets to the receivers. It calls *skipemptyspaces()* function (described below) to remove empty and negative entries from the buffer.

```
void skipemptyspaces() {
    int j=0, skip=0;
    //skip checks function removes some entries or not
    j=rear; //j is used to search packet in buffer
    while (q[j]<=0 && q[j]<last_received_pkt) {
        if (q[j]<0) skip=1;
        q[j]=0; retry_count[j]=0;
        if (diff[cid]>0) diff[cid]--;
        j++; j=j%bufferSize;
    }
    rear=j;
    if (skip>0) {
        isNack2Send=false;
        nextnack();
    }
}
```

skipemptyspaces() function performs the following actions.

- It removes the empty and lost packet entries from the buffer and updates *rear* (buffer pointer) for indicating a packet to transmit.
- *isNack2Send* variable is set to *false* because it removes lost packets and calls *nextnack ()* function (described above in T1) to check for another lost packet.

6) T6 TRANSITION

In this edge, the client broadcasts packets to the receives via *broadcast2Receivers* (broadcast channel). It calls *transmit()* function (described below) to update the client model.

```
void transmit() {
    last_send_pkt=q[rear];
    if (q[rear]==last_received_pkt)
        Send2Receivers=false;
    q[rear]=0;
    retry_count[rear]=0;
    if (diff[cid]>0) diff[cid]--;
    rear++; //cid indicates the Client ID
    rear= rear%bufferSize;
}
```

- In *transmit ()* function, the sent packet number is assigns to the *last_send_pkt* variable and *Send2Receivers* variable is sets to *false*, when there are no more packets to send. It removes the sent packet record from the buffer and updates *rear* and *diff* variables.

C. FORMAL SPECIFICATION OF THE SERVER PROCESS

The functional requirement of the server process is to send repair packets. There are four edges and one state in the model. The model for the server process is shown in Figure 6. To elaborate the figure, we show transitions by labelling the

TABLE 2. Declarations in formal specification of the server process.

<i>isPacketinBuffer</i>	It indicates the presence of NACK request(s) in the buffer.
<i>packetinbuffer</i>	It indicates the received NACK request for a packet already exists in the buffer or not.
<i>nackbuffer</i>	This buffer is used to store NACK requests to respond to them in sequence.
<i>lostPacket</i>	The server assigns a repair packet to <i>lostpacket</i> variable to send repaired packet.
<i>front and rear</i>	These are <i>q</i> buffer pointers and are used to indicate the packet to receive and transmit.
<i>frtnack and vernack</i>	These are NACK buffer pointers and are used to indicate the Nack requests.

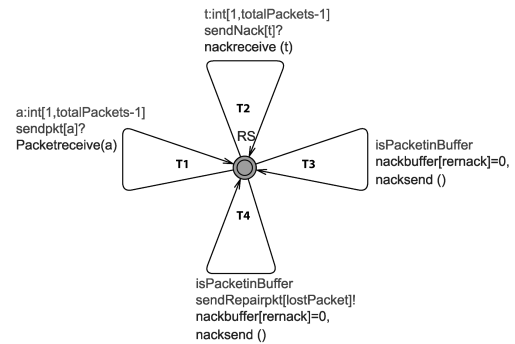


FIGURE 6. Formal specification of the server process.

edges of the server process as *T1*, *T2*, *T3* and *T4*. In Table 2, we describe the variables used in the specification of the server process.

We describe the edges as;

1) TRANSITION T1

In this edge, the server receives packets from the source via *sendpkt* (broadcast channel) and stores them in a repair buffer. It calls *Packetreceive()* function to add received packets into the buffer.

```
void Packetreceive(int p) {
    //p indicates received packets
    if (q[front]==lostPacket) nacksend ();
    q[front]=p;
    front++;
    front=front%bufferSize;
    count++; // counting number of packets
    if (count>bufferSize) rear=count%bufferSize;
}
```

Packetreceive(int p) function performs the following set of actions.

- It adds the received packet to the buffer and updates the buffer's pointers.
- It updates the record of NACK requests when a new packet is replaced with a requested packet in the buffer.

2) TRANSITION T2

In this edge, the server receives the *NACK* messages from the client(s) via *SendNack* (Unicast channel). It calls *nackreceive()* function to update the process model.

```
void nackreceive (int nackpkt) {
    packet_in_nackbuffer=false;
    pktavailable=0;
}
```

```

isAvailable(nackpkt);
if (pktavailable==1) {
    packet_in_nackbuffer=false;
    j=0; // j is used to search a packet in buffer
    while (j<bufferSize){
        if (nackbuffer[j]==nackpkt)
            packet_in_nackbuffer=true;
        j++;
    }
    if (packet_in_nackbuffer==false) {
        nackbuffer[frtnack]=nackpkt;
        frtnack++;
        frtnack=frtnack%bufferSize;
        lostPacket= nackbuffer[rearnack];
        isPacketinBuffer=true;
        receiveack=true;
    }
}

```

nackreceive (int *nackpkt*) function performs the following set of actions.

- It receives a packet as input argument to add NACK request in the buffer.
- It checks availability of requested packet in the repair buffer by calling *isAvailable* (described below) function. It performs further actions, if the packet is available in the buffer else do nothing.
- It adds NACK requests in the NACK buffer if it is not in the queue and updates the NACK buffer pointers.
- It assigns least NACK request of the buffer to the *lostpacket* variable.

```

void isAvailable(int pkt){
    int i=0;
    // i is used to search a packet in the buffer
    receiveack=true;
    while (i<bufferSize && isPacketinBuffer==false){
        if (pkt==q[i]) pktavailable++;
        i++;
    }
}

```

isAvailable(int *pkt*) function compares all packets in the repair buffer with receive a packet from *nackreceive*() function to check the availability of the requested packet. *pktavailable* variable is set to *true*, when the packet is available.

3) TRANSITION T3

This transition shows the sending of a repair packet by the server which is lost before reaching the client. The server sends packets to the clients and removes the sent packet requests from the NACK buffer by calling *nacksend*() function (described below).

```

void nacksend () {
    rearnack++;
    rearnack=rearnack%bufferSize;
    if (rearnack>=frtnack){
        isPacketinBuffer=false;
        receiveack=false;
    }
    if (rearnack<frtnack)
        lostPacket= nackbuffer[rearnack];
    else lostPacket=totalPackets;
}

```

nacksend () function perform following actions.

- It removes sent packets from the NACK buffer and updates *rear* pointer of the buffer.
- *isPacketinBuffer* variable is set to *false*, when the buffer is empty. Else it assigns the next packet of the NACK buffer to the *lostPacket* variable.

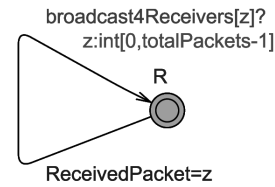


FIGURE 7. Formal specification of the receiver process.

4) TRANSITION T4

It *T4*, the server sends repair packets to the client(s) via *SendRepairPkt* channel and updates the buffer by calling *nacksend*() function (Described above in *T3*).

D. FORMAL SPECIFICATION OF THE RECEIVER PROCESS

The receivers are end users of the transmission process. Each receiver is receiving packets from its relevant client via *broadcast4Receiver*() (broadcast channel). The receiver process model is shown in Figure 7. There is one state with one edge in the model.

V. COOPERATIVE PACKET RECOVERY PROTOCOL ARCHITECTURES

As discussed in Section III, two architectures are proposed for the extended version of the protocol [7]. The architectures are shown in Figures 2 and 3. We verify both architectures of the protocol. Specification difference between both architectures is given below;

A. ARCHITECTURES 1

In architecture 1, a unicast channel is used to send repair packet(s) from the server to the client. Repair packets are only sent to the requested client. For this purpose, we send the client ID with the packet. We perform the following set of actions to model this behavior,

- The client sends its process ID with the NACK request (*sendNack[cid]* [*packet2Recover*]) when the client needs to recover a packet.
- The server receives packets from the client with client-ID (*sendNack[cid]* [*lostpacket*]) and stores packets in the *packet-ID* and client-ID in the *nack-ID* buffers.
- Repair packets are sent to the requested client with client-ID via *sendRepairpkt [id]* [*lostPacket*] channel.
- The client receives a repair packet with client's ID so the requested client can only receive the repair packet.

The number of NACK requests is greater in this model because the server independently deals with NACK requests from all clients. For example, if two clients send NACK for the same packet, the server sends repair packets one by one.

B. ARCHITECTURES 2

In architectures 2, a broadcast channel (*sendRepairpkt [lostPacket]*) is used to send repair packets from the server to the client. The server sends a repair packet to all clients when any client sends a NACK request. The number of NACK requests is less than architecture 1 because the server sends

repair packets to all clients. For example, the server sends a repair packet to all clients once when two or more clients send the NACK request for the same packet.

VI. FORMAL SPECIFICATION OF REQUIREMENTS

We label the formalise functional requirements as R1, R2, R3, and R4, which we extract from the article under study, described in section III. We express these requirements as:

1) R1

The protocol must have no deadlock while the source is sending packets. In other words, all processes will stop only if there is nothing to broadcast on the source side. We express this requirement as:

$$A[] \text{ deadlock imply Source.Packet} \approx \text{totalPackets}$$

Here *Source.Packet* is a counter in the source. Its initial value is '0' and it increases upon sending a packet. When this counter reaches the *totalPackets*, it means there are no more packets to send.

2) R2

The server must reply to all NACK requests with a repair packet to the requesting client. We express this requirement as:

$$A[] \text{ deadlock imply} \\ \text{all_available_packets_RecoverOrNot()}$$

The server process adds NACK requests in the *nackbuffer[]* and removes when replies that NACK. In this formula, we check *all_available_packets_RecoverOrNot()* function for all deadlocks. This function checks whether all the requested packets in the buffer are sent to the clients.

```
bool all_available_packets_RecoverOrNot() {
    int i;
    for (i=0; i<totalPackets; i++)
        if (nackbuffer[i]!=0)
            return false;
    return true;
}
```

3) R3

The client must send a NACK request for all lost packets before the end of the transmission. We express this requirement as,

$$A[] \text{ Client.Packet2send} == \text{Client.} \\ \text{lastReceivedPkt imply allmisspacketsOrNacked()}$$

Here, *lastReceivedPkt* indicates the most recent packet received by the client and *Packet2send* indicates the packet sent to the receivers. We call *allmisspacketsOrNacked()* function (described below) when the client multicasts its last received packet.

```
bool allmisspacketsOrNacked()
{
    int i;
    for (i=0; i<=totalPackets; i++)
        if (misspktbuffer[i]!=0)
            return false;
    return true;
}
```

TABLE 3. Results.

Requirement	Status	Computational cost	Used Memory
R1	Satisfied	82.15h	760,199,224KB
R2	Not satisfied	176.161s	3,493,784KB
R3	Not satisfied	0.015s	27,964KB.
R4	Not Satisfied	0.047s	29,156KB.

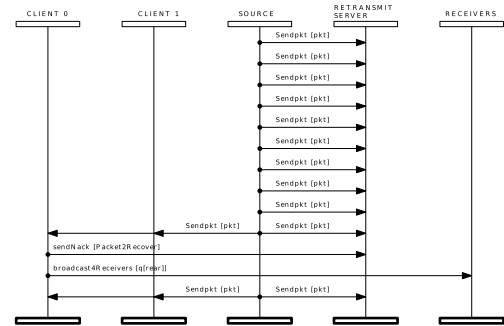


FIGURE 8. Trace for R2 requirement.

The client adds all missing packets in the *misspktbuffer* and removes them when sending a NACK request. This buffer must be empty when the client sends the last packet.

4) R4

The client must broadcast all received packets before deadlock. We express this requirement as:

$$A[] \text{ forall } (i : id_t) \text{ Client}(i).Recovered_pkt_lost < 1$$

As described in *receiveNack* function (Described in Section IV), the client adds recovered packets into the buffer. Initially, *Recovered_pkt_lost* is '0' and updated to '1' when a recovered packet is not added to the buffer.

VII. RESULTS

In this section, We apply a tool known as 'verifier', which is a part of the UPPAAL toolset for verification. Results are described in Table 3. Both architectures of the cooperative packet recovery protocols are almost similar as described in section IV so the results are the same for both architectures.

1) R1

Requirement R1 is satisfied. Deadlock can only occur when there is no more packet to send by the source, and its packet count reaches the total number of packets.

2) R2

This requirement is violated when the server receives a NACK request for a lost packet, and that packet is available in the repair buffer but is replaced by a new packet received from the source before sending the packet. There is a limited buffer size, so each packet in the repair buffer is replaced with a new packet after a specific time interval. The diagnostic trace for the requirement is shown in Figure 8. We express the figure using the playback buffer of the client and the repair buffer of the server.

Playback Buffer	-6	-7	-8	-9	10
Repair Buffer	6	7	8	9	10

As described before in Section III, the packets are placed in the buffers using

PacketSequenceModuloBufferSize

As shown in Figure 8, the first 9 packets are missed by the client. The client receives the 10th packet and sends *NACK* request for the least missing packet in the buffer. The server receives a *NACK* request for the 6th packet, and the requested packet is available in its buffer, but it receives a new packet from the source before sending the requested packet. The requested packet is replaced with the new packet as shown in the *playback* buffer.

Playback Buffer	-11	-7	-8	-9	10
Repair Buffer	11	7	8	9	10

3) R3

This requirement is violated when the client sends a packet to the receivers and there is some missing packet in the playback buffer before that packet. In that scenario, the client cannot send *NACK* request for the missing packets.

As shown in Figure 9, the 1st packet is not received by the clients and the client(0) receives 2nd packet. *Send2Receiver* flag of the client(0) becomes true by receiving a packet from the source. *IsNack2Send* also becomes true because there is a missing packet in the buffer. The client executes the edge T5 for sending a packet to the receivers before sending a NACK request for the lost packets. In this scenario, the client removes the record of lost packets and also reset the *IsNack2Send* flag.

4) R4

This requirement is violated when the client sends a *NACK* request for a lost packet and receives the lost packet after sending the greater sequence packet. The client cannot send the recovered packet to the receivers.

As shown in Figure 10, the client receives 2^{nd} packet and sends the *NACK* request for the 1^{st} packet. The client sends the 2^{nd} packet to the receivers before recovery of the 1^{st} packet. After that, the client receives the 1^{st} packet, but it cannot send this recovered packet to the receivers because it sends packets in a proper sequence.

VIII. TRANSMISSION DATA RATE (DR) DIFFERENCE BETWEEN SENDING AND RECEIVING

The client receives packets from the source and sends them to the receivers. We analyse the transmission speed i.e. A receiving (DR_{Rx}) and a transmitting (DR_{Tx}) the difference in the client process. DR expresses the transmission speed and it can be different, as described below in the relation of (DR_{Rx}) and (DR_{Tx});

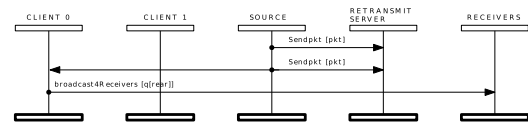


FIGURE 9. Trace for R3 requirement.

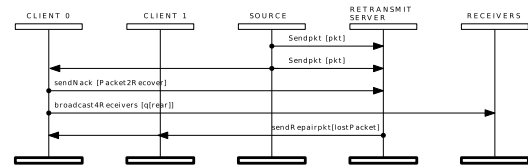


FIGURE 10. Trace for R4 requirement.

 $DR_{TX} > DR_{RX}$

This relation causes a reduction in the active part of the playback buffer. As discussed in Section III, the client sends packets to the receiver with a delay according to the active part of the buffer size. The client recovers a lost packet during this delay. So, there is no delay at the client side then it will immediately transfer packets to the receivers. In this scenario, this packet recovery mechanism is not effective because there is no time delay to recover a packet.

 $DR_{TX} < DR_{RX}$

This relation causes packet overwriting in the client's buffer. There is a limited size buffer, and every newly arrived packet must be added. With a limited size buffer, a packet can be overwritten even though it is not transmitted yet.

$$DR_{TX} \approx DR_{RX}$$

It is an ideal relation for this protocol. The active part of the buffer will remain the same for all the time.

There is no mechanism defined for complete transmission in the protocol [7]. In this protocol, authors of [7] define an inter-packet delay of 65ms in the source to the client communication to avoid packet collision. On the distributor side (the client to the receiver), the inter-packet delay is not necessary due to local distribution. In this scenario, the overall speed of DR_{Tx} is greater, so the active part of the buffer is reduced continuously during communication. This speed difference is directly proportional to the size of the inter-packet delay.

We derive a formula to find out the active part of the buffer during communication. We can observe the effect on the active part of the buffer by changing speed difference and inter-packet delay. We can adjust the buffer size according to our data size and transmission mechanism for successful communication.

We can find the effect of inter-packet delay on the active part of the buffer when there is no speed difference ($DR_{Tx} = DR_{Rx}$) using the formula provided in (1).

$$APB = \sum_{Pkt=1}^{\infty} BS - \frac{DR \times IPD}{8} \times Pkt \quad (1)$$

Here,

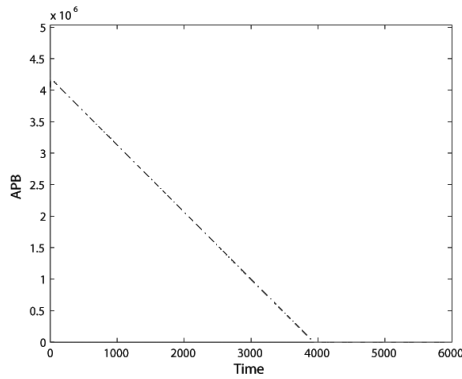


FIGURE 11. Effect of IPD on APB.

Pkt : Packet Sequence Number

BS : BufferSize

DR : DataRate

APB : ActivePartOfBuffer

IPD : Inter-PacketDelay

$\frac{DR_{Rx} \times IPD}{8}$: Convert IPD into data (bytes).

In Figure 11, we can observe the size of the active part of the buffer is continually decreasing with each data packet transmission. We extract the following basic communication parameters of transmission from the protocol to find APB :

$$\begin{aligned} \text{Buffer Size} &= \\ &= (\text{Number of Packets}) \times (\text{packet Size}) \\ &= 4096 \times 1024 = 4194304 \text{ Bytes} \\ \text{Data Rate} &= 128 \text{ Kbps} \\ \text{IPD} &= 65 \text{ ms} \\ \text{Total Data Packets} &= 6000 \text{ Packets} \end{aligned}$$

In Figure 11, we simulate Equation 1 using Matlab for the parameters given in the protocol. As shown in Figure 11, initially, the size of APB is equal to BS , but it decreases gradually and approaches zero after transmissions of some data packets. The number of data packets is shown in the time scale of the figure.

When $DR_{Tx} \neq DR_{Rx}$, we can find APB by adding the number of received packets and subtracting the number of transmitted packets from actual BS after every time unit (Sec). We can calculate the required DR and IPD , according to data size for successful communication using (2). We need to maintain the size of APB , almost equal to BS to recover missing packets. The size of APB must be greater than the client's turnaround time to the server for the packet recovery process.

$$APB = \sum_{T=1}^{\infty} BS - \frac{T}{8 \times PS} \quad (2)$$

Values of α and β are described bellow,

$$\begin{aligned} \alpha &= \frac{Tpkt_{Rx} + IPD_{Rx}}{Tpkt_{Rx}} \\ \beta &= \frac{Tpkt_{Tx} + IPD_{Tx}}{Tpkt_{Tx}} \end{aligned}$$

$Tpkt_{Rx}$ = Time required to receive one packet.

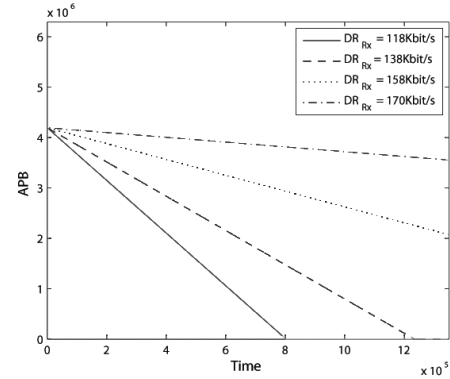


FIGURE 12. Effect of the Speed difference and IPD on APB.

$Tpkt_{Tx}$ = Time required to transmit one packet.

In Figure 12, we simulate equation 2 using Matlab. Here, the (DR_{Tx}) is constant (128kbps), and we plot the graph for multiple values of the (DR_{Rx}) with some constant IPD . We can observe from the figure that APB can be maintained nearly equal to the BS by selecting optimal values of (DR_{Rx}) , (DR_{Tx}) , and IPD .

IX. LIMITATION AND CHALLENGES

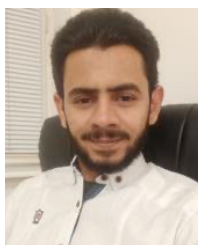
The limited size of the computer's main memory is the main limitation in the model checking using the UPPAAL toolset. The system model generates an ample state space. It causes a state space explosion problem, a big challenge to modeling a system. In this article, we verify the extended architecture of cooperative recovery protocol. We design a system model with two clients (Client(0) and Client(1) and three receivers (Receiver(0), Receiver(1) and Receiver(2) for each client because of limited main memory. We verify the system model for the limited size of packets and limited buffer size in the clients and the server. The functional properties of the system models are not affected by these limitations because limited data and buffer size, along with few numbers of clients and receivers, can reflect the whole behaviour of the protocol.

X. CONCLUSION

In the cooperative packet recovery protocol, Nicholas F. Maxemchuk et al. claim QoS improvements in multicast communication. They introduce a server, a closed link with the source. This server stores packets in its buffer and recovers a packet if the packet is reported as lost. We formally verify their claim by specifying the protocol for four basic components (Source, Server, Client, and Receivers) using the UPPAAL toolset. We extract the protocol's functional requirements and formalise them [7]. We verify the system model for these formalise function requirements and report the scenarios where these requirements are unsatisfied by using a message sequence chart. We design a mathematical model to determine the size of the active part of the buffer in communication. We also express the effects of inter-packet delay and speed difference on the active part of the buffer using MATLAB graphs.

REFERENCES

- [1] I. Sodagar, "The MPEG-DASH standard for multimedia streaming over the internet," *IEEE MultimediaMag.*, vol. 18, no. 4, pp. 62–67, Apr. 2011, doi: [10.1109/MMUL.2011.71](https://doi.org/10.1109/MMUL.2011.71).
- [2] S. Brown, "Networked system for interactive communication and remote monitoring of individuals," U.S. Patent 7 921 186, Apr. 5, 2011. [Online]. Available: <https://www.google.com/patents/US7921186>
- [3] M. Hosseini, Y. Jiang, R. R. Berlin, L. Sha, and H. Song, "Towards physiology-aware DASH: Bandwidth-compliant prioritized clinical multimedia communication in ambulances," 2017, *arXiv:1707.04918*.
- [4] Y. Al-Ali and J. Wafeeq, "Systems and methods for monitoring a patient health network," USA Patent 4 198 350, Sep. 18, 2014. [Online]. Available: <https://www.google.com/patents/US20140266790>
- [5] Q. Shen, X. Liang, X. Shen, X. Lin, and H. Y. Luo, "Exploiting geo-distributed clouds for a E-health monitoring system with minimum service delay and privacy preservation," *IEEE J. Biomed. Health Informat.*, vol. 18, no. 2, pp. 430–439, Mar. 2014.
- [6] J. Xu and B. W. Wah, "Delay-aware loss-concealment strategies for real-time video conferencing," in *Proc. IEEE Int. Symp. Multimedia (ISM)*, Dana Point, CA, USA, Dec. 2011, pp. 27–34, doi: [10.1109/ISM.2011.14](https://doi.org/10.1109/ISM.2011.14).
- [7] N. F. Maxemchuk, K. Padmanabhan, and S. Lo, "A cooperative packet recovery protocol for multicast video," in *Proc. Int. Conf. Netw. Protocols*, 1997, pp. 259–266.
- [8] C. Baier and J. Katoen, *Principles of Model Checking* (Representation and Mind Series). Cambridge, MA, USA: MIT Press, 2008.
- [9] G. Behrmann, A. David, and K. Larsen, "A tutorial on UPPAAL," in *Formal Methods for the Design of Real-Time Systems*. Cham, Switzerland: Springer, 2004, pp. 200–236.
- [10] S. Cranen, J. F. Groote, J. J. A. Keiren, F. P. M. Stappers, E. P. de Vink, W. Wesselink, and T. A. C. Willemse, "An overview of the MCRL2 toolset and its recent advances," in *Proc. 19th Int. Conf., TACAS Held Eur. Joint Conf. Theory Pract. Software (ETAPS)* (Lecture Notes in Computer Science), Rome, Italy, N. Piterman and S. A. Smolka, Eds. Cham, Switzerland: Springer, vol. 7795, 2013, pp. 199–213, doi: [10.1007/978-3-642-36742-7_15](https://doi.org/10.1007/978-3-642-36742-7_15).
- [11] H. Garavel, F. Lang, R. Mateescu, and W. Serwe, "CADP 2011: A toolbox for the construction and analysis of distributed processes," *Int. J. Softw. Tools Technol. Transf.*, vol. 15, no. 2, pp. 89–107, Apr. 2013, doi: [10.1007/S10009-012-0244-Z](https://doi.org/10.1007/S10009-012-0244-Z).
- [12] G. J. Holzmann, *The SPIN Model Checker—Primer and Reference Manual*. Reading, MA, USA: Addison-Wesley, 2004.
- [13] M. Atif, "Formal modeling and verification of distributed failure detectors," Faculty Math. Comput. Sci., Eindhoven Univ. Technol., The Netherlands, vol. 10, Tech. Rep., 2011.
- [14] B. Bordbar, R. Anane, and K. Okano, "An evaluation mechanism for QOS management in wireless systems," in *Proc. Parallel Distrib. Syst. 11th Int. Conf.*, vol. 2, 2005, pp. 150–154.
- [15] P. Pettersson, *Modelling and Verification of Real-Time Systems Using Timed Automata: Theory and Practice*. Sweden: Department of Computer systems, Uppsala Univ., 1999.
- [16] M. Atif and M. Mousavi, "Formal specification and analysis of accelerated heartbeat protocols," in *Proc. Summer Comput. Simul. Conf. Soc. Comput. Simul. Int.*, 2010, pp. 403–412.
- [17] Y. B. Kartal, E. G. Schmidt, and K. W. Schmidt, "Modeling distributed real-time systems in TIOA and UPPAAL," *ACM Trans. Embedded Comput. Syst.*, vol. 16, no. 1, pp. 22:1–22:26, 2016.
- [18] F. Ansgar, V. Lodewijk, and M. Angelika, "Modelling and verification of the Imac protocol for wireless sensor networks," in *Integrated Formal Methods*, vol. 4591, no. 1, J. Davies and J. Gibbons, Eds. Essen, Germany: IFM, 2007, pp. 253–272, 2007.



MUHAMMAD NAEEM received the master's degree in computer science from The University of Lahore, Lahore, Pakistan. He was a Lecturer with The University of Lahore, from 2015 to September 2019. He is currently a Ph.D. Research Fellow with the Department of Computer Science, Aalborg University, Denmark. His research interest includes statistical model checking of distributed systems.



MUHAMMAD ATIF (Senior Member, IEEE) received the M.Sc. degree in computer science from Gomal University, Dera Ismail Khan, in 1997, and the Ph.D. degree in computer science from Eindhoven University of Technology, The Netherlands, in 2011. In 2007, he won a five-year scholarship from the Higher Education Commission of Pakistan for studies of the M.S. degree leading to the Ph.D. degree. He joined the Department of Computer Science, COMSATS Lahore, as an Assistant Professor, in 2011, and he shifted to The University of Lahore, in 2014, and is still working there. He is currently an Associate Professor. His research interest includes the formal verification of distributed algorithms.



ARSHAD ALI received the B.Sc. degree in mathematics and physics and the M.Sc. degree in computer science from Punjab University, Lahore, in 1997 and 2003, respectively, the M.Phil. degree in information technology (informatique), in 2009, with a focus on qualification of specialist in mobile networks, and the Ph.D. degree in information technology (telecommunications and electronics) jointly from the Institute of Telecom SudParis and UPMC (Paris VI). He was a Postdoctoral Researcher with Orange Laboratories, Paris. From 2013 to 2021, he was with the Department of Computer Science, The University of Lahore. He is currently an Associate Professor with the Department of Computer Science, National University of Computer Science and Emerging Sciences, Lahore Campus, Pakistan. His research interests include mobile ad-hoc networks, network coding, natural language processing, machine learning, deep learning, and image processing in healthcare.



MARYAM GULZAR received the B.S. degree in computer science from the Department of CS, The University of Lahore (UOL), Pakistan, in 2016, and the M.S. degree in SPM with the School of Computing, National University of Computer and Emerging Sciences (NUCES), Lahore Campus, Pakistan, in 2018. She was a Lecturer with the Department of Software Engineering, UOL. She is currently undertaking the Ph.D. degree as a Junior Researcher with the Department of Software Engineering, LUT University, Finland. Her research interests include digital transmission, information systems, and machine learning. She won a research grant from the Higher Education Commission of Pakistan for four years to complete the Ph.D. studies.



IMRAN RIAZ HASRAT received the master's degree in computer science from The University of Lahore, Pakistan. He was a Software Engineer with Goodlogics Pvt., Lahore. He was a Lecturer of computer science with The University of Lahore. He is currently a Ph.D. Research Fellow with the Department of Computer Science, Aalborg University, Denmark. His research interests include implementing formal methods and machine learning-based tools for cost optimization

of residential heating control solutions, energy-aware buildings, strategy synthesis, and multi-objective optimization.

...